TabularAML's genetic feature engine: a deep technical audit and upgrade roadmap

TabularAML's automatic feature generator is a **well-architected evolutionary search** over row-wise numeric and categorical transformations, with notably strong leakage prevention and sophisticated stagnation handling — but it operates in a search paradigm that is now one full generation behind the state of the art. The system's core bottleneck is its **per-candidate full-CV evaluation**, which limits throughput to ~144K model trainings at extreme budget, while OpenFE's FeatureBoost technique evaluates equivalent candidate volumes 10–100× faster via incremental residual scoring. The operator set completely lacks **group-by aggregation features** — the single most powerful feature class in Kaggle winning solutions and critical for DataCrunch 2's cross-sectional stock panel structure. The greedy forward-selection mechanism and same-fold evaluation/selection introduce well-documented statistical biases (Cawley & Talbot, 2010) that likely cause the framework to overfit its feature set to validation noise. These gaps are addressable. The genetic infrastructure (adaptive stagnation, SHAP-guided parent selection, pattern memory) is genuinely novel relative to published frameworks, and most high-impact improvements can be integrated as modular additions without architectural rewrites.

Part 1: The genetic algorithm under the hood

Population model and search loop

The `FeatureGenerator.search()` method drives an outer loop over `n_generations` (up to 80 in extreme mode). Each generation maintains a population of `Feature` objects carrying a `weight` (importance score), `depth` (transformation nesting), parent lineage, and the operation that created them. The generation list grows monotonically — accepted features are never removed except during intelligent pruning events.

The per-generation cycle is: (1) sample parents via `_sample_parents_with_memory`, (2) generate children via `_sample_children_with_creativity`, (3) rank candidates with `ImprovedAdaptiveController.rank_candidates_with_memory`, (4) evaluate top candidates greedily in `_select_elites`, (5) update adaptive state and check stagnation.

Parent sampling is the most sophisticated component. It detects "feature families" by splitting names on binary operator separators, enforcing diversity across families. For binary operations, SHAP interaction values (from `FeatureImportanceAnalyzer`) select pairs with measured synergy. A usage penalty discourages repeated selection of the same parents, and the `AdaptiveController`'s parent quality scores (success-rate as parent, from `feature_as_parent_success/attempts`) weight the sampling distribution.

Child generation in `_sample_children_with_creativity` uses **softmax-temperature sampling** over operations, where the temperature increases with stagnation level. During stagnation, it injects "creativity" by prioritizing under-used operations with low failure rates — retrieved via the controller's `get_creative_operations()` method. The creativity mechanism examines `op_stats[dtype, op_type, op]` and returns operations whose usage count is below median and whose consecutive failure count is below 3.

Candidate ranking via `rank_candidates_with_memory` computes a composite score: `weight × w_weight + op_priority × w_op + parent_quality × w_parent + novelty × w_novelty + pattern_sim × w_pattern - complexity_penalty`. Novelty is `1/(1 + failed_interactions[combination])`. Pattern similarity checks against `successful_patterns` (last 20 (parents, op, gain) tuples). The weighting vector shifts during stagnation: novelty and pattern weights increase, raw importance weight decreases — a principled exploration-exploitation tradeoff.

The five-level stagnation machine

The `StagnationLevel` enum (NONE → MILD → MODERATE → SEVERE → CRITICAL) triggers at **1/2/4/8 generations without new features** or **2/4/6/12 generations without any score improvement**. Each level maps to:

Level	Exploration intensity	Min gain multiplier	Special mechanisms
NONE	0.0	1.0×	Normal operation
MILD	0.3	0.75×	Temperature increase
MODERATE	0.6	0.5×	Creative ops prioritized
SEVERE	1.0	0.25×	Hopeful monsters activated
CRITICAL	1.5	0.1×	Partial restart + monsters

`_creative_hopeful_monster` — active at SEVERE/CRITICAL — generates candidates through four strategies: completely random parent-op combinations, deliberate use of under-used ops, multi-step transformations (applying ops to already-transformed features, increasing depth), and forcing unused parents into combinations. `_partial_restart` at CRITICAL level retains top features scored by `parent_quality + weight + successful_children_count + original_feature_bonus`, clears blacklists and usage counters, but preserves `successful_patterns` — allowing learned knowledge to survive the restart.

`_intelligent_pruning` triggers after 5+ stagnant generations. It builds a dependency graph (which features are parents of which others), protects features with successful children, and

removes the worst by importance. Features pruned multiple times get blacklisted permanently. This prevents the search space from growing unboundedly while preserving productive lineages.

Memory and learning subsystems

The `ImprovedAdaptiveController` maintains several learning structures:

- **`op_stats`** keyed by `(dtype, op_type, op_name)`: exponentially-decayed `success_rate`, `avg_gain`, `consecutive_failures`. This creates a bandit-like per-operator learning signal.
- **`failed_interactions`** counter: per-combination tracking that penalizes retrying known-bad combos. This is essentially a taboo list common in metaheuristics.
- **`feature_as_parent_success/attempts`**: parent quality scores enabling the search to route through productive feature lineages.
- **`strategy_success/attempts`**: tracks hopeful_monster vs normal strategy performance, allowing meta-level strategy adaptation.
- **`successful_patterns`** (capped at 20): recent (parent_features, operation, gain) tuples used for pattern-matching in candidate ranking.

This memory architecture is **more sophisticated than any published automated FE framework** I found in the literature. OpenFE has no memory across candidates; CAAFE maintains only a text-based history in the LLM prompt; AutoFeat has no iterative learning. The closest parallel is the policy network in FETCH (ICLR 2023), but TabularAML's explicit symbolic memory is more interpretable and doesn't require neural network pre-training.

Operator set: strong on math, missing on aggregation

The 23 unary numeric ops span the full trigonometric, logarithmic, and activation-function space: neg, abs, square, sqrt, log, log1p, exp, inv, cube, sin, cos, tan, sigmoid, tanh, reciprocal_sqrt, cbrt, floor, ceil, round, sign, arcsin, arccos, arctan. The 19 binary ops cover arithmetic (add, sub, mul, div), robust alternatives (absdiff, diff_ratio, logmul), and compositional features (geometric_mean, harmonic_mean, relative_diff, log_ratio, angle_between, weighted_sum, weighted_diff, pow, mod, max, min).

Overflow protection uses `np.clip` and `np.where` masks throughout `NUM_OPS_LAMBDAS`. Every operation is pure row-wise — no operation reads from other rows. This is a **deliberate design choice for leakage prevention**: since no operation aggregates across rows, no operation can leak target information even when applied outside CV folds.

Categorical ops are minimal: target encoding, frequency encoding, count encoding (all pipeline-required, meaning they go through `CategoricalEncoder` inside CV), and concat (for creating interaction categoricals).

Critical gap: The operator set has **zero group-by aggregation operators** (no `groupby(cat)[num].mean/std/count/min/max`). In DataCrunch 2, where `Feature_Industry` classifies stocks and cross-sectional relationships are the primary alpha signal, aggregation features like "this stock's value relative to its industry mean" are the single most predictive feature class. OpenFE includes 6 GroupBy operators; Featuretools is built entirely around aggregation primitives. This is TabularAML's largest capability gap.

Leakage prevention assessment

The leakage model is **correct and well-implemented**:

1. **Target/count/freq encoding**: Marked as `pipeline_required=True`, routed through `CategoricalEncoder` which wraps `category_encoders.TargetEncoder` and `CountEncoder`. These fit only on the training fold inside `cross_val_score`'s CV loop. The `PipelineWrapper` chains encoder → model, ensuring each fold sees only its own encoding.
2. **Row-wise ops**: Since `NUM_OPS_LAMBDAS` contains only element-wise math (no rolling windows, no group statistics, no rank transforms), features generated outside the pipeline cannot leak. This is more conservative than OpenFE, which includes GroupByThenMean as a standard operator — OpenFE explicitly warns it "cannot handle time series data" because those aggregations can leak future information.
3. **Group-aware CV**: `_groups_active` propagates group labels through to `GroupKFold` or `TimeSeriesSplit`, preventing same-group contamination.
4. **Search subsample**: The `search_sample_size` (10–15K rows) uses stratified/group-aware sampling, avoiding bias in the search population.

One subtle issue: The same CV folds used to evaluate whether a feature provides gain are also used to report the improvement. This is a classic "selection from CV validation" bias (Ambroise & McLachlan, 2002). Over 80 generations with hundreds of candidates per generation, the cumulative effect of selecting the "best-on-CV" features can produce **several percentage points of optimistic bias**, particularly on small datasets. The feature set is overfitted to the CV split pattern even though individual features are correctly cross-validated.

Fitness evaluation cost

In extreme mode: **80 generations × 360 children × 5 CV folds = 144,000 XGBoost training runs**. Each trains on `search_sample_size` rows (~10–15K). At ~0.5–2 seconds per fit on CPU, the total search takes **20–80 hours**. This is the framework's primary scalability constraint.

Contrast with OpenFE's FeatureBoost: evaluating a candidate feature requires training a **single-feature LightGBM model** on residuals with `init_score` — roughly **50–100ms per candidate**. For the same 144K budget, OpenFE evaluates ~10× more candidates, and each evaluation is ~10×

cheaper. The two-stage successive halving further concentrates compute on promising candidates.

Where TabularAML will succeed and struggle

Strengths in practice: The framework excels on **moderate-dimensional tabular datasets (20–200 features)** where the search space of pairwise row-wise transformations is tractable. The adaptive stagnation handling prevents premature convergence better than simpler genetic algorithms. The SHAP interaction-guided parent selection directs search toward genuinely synergistic pairs. For **DataCrunch 2 specifically**, the row-wise ops are safe against temporal leakage — a major advantage over OpenFE, which would naively compute GroupByThenMean across all moons.

Weaknesses: On high-cardinality panel data (like DataCrunch 2 with ~3000 stocks × ~200 moons), the most valuable features are cross-sectional aggregations, not row-wise transforms. The search budget is consumed by expensive per-candidate CV, leaving less room for exploration than proxy-based methods. Greedy forward-selection can miss complementary feature sets (suppressor variables, XOR-type interactions). The framework cannot discover temporal patterns (lags, rolling statistics) that are standard in quant finance.

Part 2: Where TabularAML stands against the field

Search strategy comparison

Framework	Strategy	Candidates/hour (est.)	Leakage safety	Memory/learning
TabularAML	Genetic algorithm + adaptive stagnation	~2K (full CV)	Excellent (row-wise only)	Extensive (op_stats, patterns, parent quality)
OpenFE	Exhaustive + FeatureBoost + successive halving	~50K–200K (residual scoring)	Moderate (GroupBy ops can leak in time-series)	None across candidates
CAAFE	LLM-guided iterative	~20–50 (full retrain + LLM latency)	Moderate (LLM can propose leaky features)	Text history in prompt

Framework	Strategy	Candidates/hour (est.)	Leakage safety	Memory/learning
AutoFeat	Exhaustive + L1 regularized selection	All-at-once (batch LASSO)	Weak (no temporal handling)	None
Featuretools	Deterministic DFS over entity relationships	All-at-once (enumeration)	Strong (cutoff times)	None
LLM-FE (2025)	Evolutionary + LLM mutation/crossover	~100–500 (LLM + eval)	Moderate	Island model populations
OCTree (NeurIPS 2024)	LLM + decision tree feedback	~50–200	Moderate	DT reasoning feedback

TabularAML's genetic approach occupies a **middle ground**: more directed than exhaustive enumeration (AutoFeat), more scalable than LLM-based methods (CAAFE, OCTree), but dramatically slower per-candidate than OpenFE's FeatureBoost. The memory/learning subsystem is TabularAML's strongest differentiator — no published framework maintains comparable per-operator statistics, parent quality tracking, or pattern memory.

Operator set comparison

OpenFE's operator set is the most directly comparable. OpenFE includes **6 GroupBy operators** (Mean, Std, Median, Min, Max, Rank) that TabularAML entirely lacks. These operators are responsible for OpenFE's strongest results on datasets with categorical grouping structure. Conversely, TabularAML has **13 more unary ops** (the full trigonometric + activation function suite) and **12 more binary ops** (geometric_mean, harmonic_mean, log_ratio, angle_between, weighted_sum, etc.). For pure numeric interaction discovery, TabularAML's operator breadth is superior. But for structured tabular data with categories, OpenFE's GroupBy operators deliver more predictive power per feature.

Featuretools operates in a completely different paradigm — its "deep" feature synthesis stacks aggregation primitives across entity relationships, creating features like `MEAN(orders.SUM(items.price))`. This relational depth is orthogonal to TabularAML's approach and would require architectural changes to replicate.

CAAFE and LLM-FE have **unrestricted operator sets** (arbitrary Python code), which means they can propose domain-specific features (date decomposition, domain ratios) that no fixed operator set covers. The ELF-Gym benchmark (CIKM 2024) found that LLMs capture ~56% of

expert Kaggle features semantically but only 13% at implementation level, suggesting fixed operator sets with broader coverage often outperform LLM proposals in practice.

Evaluation method: TabularAML's biggest efficiency gap

This is where the performance gap is most stark. TabularAML evaluates each candidate via **full cross_val_score** — training a complete XGBoost model on all existing features plus the candidate, across 5 folds. This is the **gold standard for accuracy** but the **worst case for throughput**.

OpenFE's FeatureBoost trains a LightGBM on just the candidate feature(s) against base model residuals. The `init_score` parameter in LightGBM makes this equivalent to evaluating the marginal contribution of the new feature on top of the base model, without retraining the base model. This is **mathematically sound** (it's exactly the next boosting step) and **10–100× faster**.

The successive halving in OpenFE's Stage 1 further amplifies efficiency: candidates are first evaluated on 1/2^q of the data, and only the top half advance to larger data subsets. This concentrates compute where it matters.

Net effect: For the same compute budget, OpenFE explores a search space ~100–1000× larger than TabularAML. In their paper, OpenFE evaluates millions of candidates on datasets with hundreds of features — TabularAML would require months for the same coverage.

Benchmark context

OpenFE reports beating 99.3% of 6,351 teams on IEEE-CIS Fraud Detection (Kaggle) and 99.6% on another competition, with **1.9% average accuracy improvement** across 49 OpenML datasets. These are strong results. TabularAML has no published benchmarks, but the Google Drive competition strategy document describes it as providing an "insurmountable competitive advantage" when paired with AutoGluon — suggesting comparable real-world effectiveness on the competitions where it's been deployed.

The 2025 usability survey of 53 AutoFE methods found most are "hard to use, lack documentation, no active communities." OpenFE and CAAFE are the only two that are pip-installable and functional. TabularAML's integrated pipeline approach (encoding → generation → evaluation → selection in one `search()` call) is a usability advantage for the practitioner.

What TabularAML does genuinely better

Three aspects stand out relative to the published literature:

1. **Adaptive stagnation handling with 5 escalation levels** — no other framework has this. Most either run for a fixed budget (OpenFE, AutoFeat) or use simple early stopping. The graduated response (temperature → creative ops → hopeful monsters → partial restart) is a principled metaheuristic design.

2. **SHAP interaction-guided parent selection** — using measured feature synergy to select binary operation parents is more directed than random or importance-weighted pairing. OpenFE pairs features by enumeration; CAAFE relies on LLM intuition. Neither uses empirical interaction measurements.
3. **Leakage-safe operator design** — by restricting to row-wise operations and routing encodings through the CV pipeline, TabularAML achieves the strongest leakage guarantees of any framework. OpenFE's GroupBy operators and CAAFE's unrestricted Python code both create leakage risks on panel/time-series data.

Part 3: Concrete upgrade path to state-of-the-art performance

Priority 1: FeatureBoost-style proxy evaluation (expected impact: HIGH)

Why: This single change would increase candidate throughput by ~50×, enabling exploration of the search space that currently requires days in hours. OpenFE's core innovation demonstrates that residual-based scoring correlates highly with full-CV evaluation while being orders of magnitude faster.

Where to modify: `features.py` → `_select_elites()` and the evaluation calls within `search()`.

Integration pattern:

```python
```

```
# In FeatureGenerator, add a method:
def _train_base_model_and_get_residuals(self, X, y, cv):
    """Train base model on current features, return OOF predictions."""
    import lightgbm as lgb
    oof_preds = np.zeros(len(y))
    for train_idx, val_idx in cv.split(X, y, groups=self._groups):
        dtrain = lgb.Dataset(X.iloc[train_idx], y.iloc[train_idx])
        model = lgb.train({"objective": self._objective, "verbosity": -1,
                           "n_estimators": 200, "learning_rate": 0.1},
                          dtrain, num_boost_round=200)
        oof_preds[val_idx] = model.predict(X.iloc[val_idx])
    return oof_preds

def _featureboost_score(self, candidate_values, y, oof_preds, cv):
    """Score a single candidate feature via residual-based incremental training."""
    import lightgbm as lgb
    scores = []
    for train_idx, val_idx in cv.split(candidate_values, y, groups=self._groups):
        dtrain = lgb.Dataset(
            candidate_values.iloc[train_idx].values.reshape(-1, 1),
            y.iloc[train_idx],
            init_score=oof_preds[train_idx]  # KEY: base model predictions as offset
        )
        dval = lgb.Dataset(
            candidate_values.iloc[val_idx].values.reshape(-1, 1),
            y.iloc[val_idx],
            init_score=oof_preds[val_idx],
            reference=dtrain
        )
        model = lgb.train(
            {"objective": self._objective, "num_leaves": 16,
             "n_estimators": 50, "verbosity": -1},
            dtrain, valid_sets=[dval],
            callbacks=[lgb.early_stopping(10, verbose=False)]
        )
        # Score improvement = base_loss - (base_loss + residual_model)
        base_score = self._metric(y.iloc[val_idx], oof_preds[val_idx])
        new_score = self._metric(
            y.iloc[val_idx],
            oof_preds[val_idx] + model.predict(candidate_values.iloc[val_idx].values
        )
```

```
        scores.append(new_score - base_score)
    return np.mean(scores)
```

Two-phase search: Use FeatureBoost as a **fast pre-filter** (evaluate all candidates cheaply), then full-CV only for the top 5–10% of candidates. In `search()`, modify the loop:

python

```
# Phase 1: FeatureBoost screening (fast)
oof_preds = self._train_base_model_and_get_residuals(X_current, y, cv)
fb_scores = {c: self._featureboost_score(c.values, y, oof_preds, cv)
             for c in all_candidates}
top_candidates = sorted(fb_scores, key=fb_scores.get, reverse=True)[:n_children // 1

# Phase 2: Full CV validation (expensive, only top candidates)
accepted = self._select_elites(top_candidates, X_current, y, ...)
```

Expected throughput gain: ~50× more candidates evaluated per generation. **Implementation complexity**: 2–3 days. Requires adding LightGBM as a dependency (likely already available given XGBoost is used).

Priority 2: Group-by aggregation operators (expected impact: HIGH)

Why: GroupBy features are the **single most powerful feature class** in Kaggle winning solutions (Chris Deotte's 1st-place solutions routinely generate 10,000+ groupby candidates). For DataCrunch 2, features like "stock's feature value relative to industry mean" capture cross-sectional structure that row-wise ops cannot.

Where to modify: `ops.py` → add to `OPS` dict and create `AGG_OPS_LAMBDAS`; `features.py` → `_sample_children_with_creativity` to generate groupby candidates; `encoders.py` → new `GroupByEncoder` for pipeline-required aggregations.

Leakage-safe implementation: GroupBy aggregations must be computed inside CV folds (they use population statistics). Add them as `pipeline_required=True`:

python

```python
# In ops.py, add:
AGG_OPS = {
    "groupby_mean": lambda df, cat_col, num_col:
        df.groupby(cat_col)[num_col].transform("mean"),
    "groupby_std": lambda df, cat_col, num_col:
        df.groupby(cat_col)[num_col].transform("std"),
    "groupby_median": lambda df, cat_col, num_col:
        df.groupby(cat_col)[num_col].transform("median"),
    "groupby_min": lambda df, cat_col, num_col:
        df.groupby(cat_col)[num_col].transform("min"),
    "groupby_max": lambda df, cat_col, num_col:
        df.groupby(cat_col)[num_col].transform("max"),
    "groupby_count": lambda df, cat_col, num_col:
        df.groupby(cat_col)[num_col].transform("count"),
    "groupby_rank": lambda df, cat_col, num_col:
        df.groupby(cat_col)[num_col].transform("rank", pct=True),
    "groupby_zscore": lambda df, cat_col, num_col:
        (df[num_col] - df.groupby(cat_col)[num_col].transform("mean")) /
        (df.groupby(cat_col)[num_col].transform("std") + 1e-8),
}

# In encoders.py, add GroupByEncoder:
class GroupByEncoder:
    """Fit-transform group-by statistics within CV folds to prevent leakage."""
    def __init__(self, cat_col, num_col, agg_func):
        self.cat_col = cat_col
        self.num_col = num_col
        self.agg_func = agg_func
        self.mapping_ = None

    def fit(self, X, y=None):
        self.mapping_ = X.groupby(self.cat_col)[self.num_col].agg(self.agg_func)
        return self

    def transform(self, X):
        result = X[self.cat_col].map(self.mapping_)
        # Handle unseen categories with global statistic
        global_val = self.mapping_.mean() if self.agg_func != "count" else 0
        return result.fillna(global_val)
```

For DataCrunch 2 specifically, the `Feature_Industry` column is the obvious groupby key. Features like `groupby_zscore(Feature_Industry, gordon_Feature_1)` — "how unusual is this stock's gordon_Feature_1 relative to its industry" — are the cross-sectional alpha signals the

competition rewards. The orthogonalization step in DataCrunch's scoring already neutralizes raw industry effects, so **industry-relative features directly target the alpha residual**.

Implementation complexity: 3–4 days (including pipeline integration and testing). The `Feature` dataclass needs a new field for `aggregation_type` and the child generation logic needs to handle the (cat_col, num_col, agg_func) triple.

Priority 3: Fix the CV selection bias (expected impact: MEDIUM-HIGH)

Why: Ambroise & McLachlan (2002) demonstrated that using the same CV folds for selection and evaluation produces near-zero error rates even on **scrambled labels** when many candidates are tested. Cawley & Talbot (2010) showed the degradation from overfitting the model selection criterion can be "comparable in magnitude to differences between learning algorithms." Over 80 generations with hundreds of candidates each, TabularAML is particularly susceptible.

Where to modify: `features.py` → `search()` method, `_select_elites()`.

Two complementary fixes:

```python
# Fix 1: Held-out meta-validation split
# In search(), before the generation loop:
if self.meta_validation:
    # Reserve 15-20% of data as meta-validation, never seen during search
    meta_idx = stratified_sample(len(X), frac=0.2, y=y, groups=groups)
    search_idx = ~meta_idx
    X_search, y_search = X.iloc[search_idx], y.iloc[search_idx]
    X_meta, y_meta = X.iloc[meta_idx], y.iloc[meta_idx]
    # All search happens on X_search; final feature set validated on X_meta
    # After search completes, re-evaluate all selected features on meta split

# Fix 2: Rotate CV folds across generations
# In search(), at each generation:
if generation % fold_rotation_period == 0:
    cv = self._create_cv(n_splits=self.n_splits, random_state=generation)
    # Different fold assignment prevents features from overfitting
    # to a specific fold pattern
```

The meta-validation split is the more robust fix. After the full search completes, retrain on the search portion and evaluate the entire feature set on the meta portion. If the meta-validation score is substantially lower than the search CV score, the feature set is overfit. Use the gap as a regularization signal to prune aggressively.

Expected impact: Prevents 1–3% of spurious gain from being counted as real improvement on small/medium datasets. Critical for DataCrunch 2 where the OOS evaluation (live market returns) is the true test. **Implementation complexity**: 1–2 days.

Priority 4: Batch multi-feature evaluation (expected impact: MEDIUM)

Why: Pure greedy forward selection fails on suppressor variables (features useless alone but valuable in combination) and correlated feature groups (selects one and misses the complementary set). Evaluating features in batches partially addresses this.

Where to modify: `features.py` → `_select_elites()`.

python

```python
def _select_elites_batch(self, candidates, X, y, batch_size=5):
    """Evaluate candidates in batches, using permutation importance
    to identify the best subset within each batch."""
    from sklearn.inspection import permutation_importance

    accepted = []
    for batch in chunk(candidates, batch_size):
        # Add all batch features to current X
        X_augmented = pd.concat([X] + [c.values for c in batch], axis=1)

        # Train model once on augmented features
        model = self._fit_model(X_augmented, y)

        # Permutation importance identifies which batch features help
        perm_imp = permutation_importance(model, X_augmented, y,
                                          n_repeats=5, scoring=self.scorer)

        # Keep batch features with positive importance above threshold
        for i, candidate in enumerate(batch):
            col_idx = len(X.columns) + i
            if perm_imp.importances_mean[col_idx] > self.adaptive_min_gain:
                accepted.append(candidate)
                X = pd.concat([X, candidate.values], axis=1)

    return accepted
```

This trains **1 model per batch** instead of 1 per candidate, reducing total evaluations by `batch_size`× while allowing complementary features to be discovered together. **Implementation complexity**: 1–2 days.

Priority 5: Feature value caching (expected impact: MEDIUM)

Why: Currently, each candidate feature's column values are recomputed from parent columns every time they're needed. When the same parent pair with the same operation appears in multiple generations (or in elite re-evaluation), the computation is wasted.

Where to modify: `features.py` → add a `FeatureCache` class.

```python
import hashlib

class FeatureCache:
    def __init__(self, max_size_mb=2000):
        self._cache = {}
        self._max_bytes = max_size_mb * 1024 * 1024
        self._current_bytes = 0

    def _key(self, parent_names, op_name):
        return hashlib.md5(f"{sorted(parent_names)}_{op_name}".encode()).hexdigest()

    def get_or_compute(self, parent_names, op_name, compute_fn):
        key = self._key(parent_names, op_name)
        if key in self._cache:
            return self._cache[key]
        result = compute_fn()
        nbytes = result.nbytes if hasattr(result, 'nbytes') else 0
        if self._current_bytes + nbytes < self._max_bytes:
            self._cache[key] = result
            self._current_bytes += nbytes
        return result
```

Integrate into child generation: before computing `op(parent1, parent2)`, check the cache. With 360 children per generation over 80 generations, cache hit rates above 30% are expected due to the genetic algorithm's tendency to revisit productive regions. **Implementation complexity**: 0.5 days.

Priority 6: GPU-accelerated evaluation (expected impact: MEDIUM)

Why: The user works with CUDA. XGBoost's `tree_method='gpu_hist'` provides **5–20× training speedup**. Combined with cuDF for feature computation, the total search time compresses proportionally.

Where to modify: `cv.py` → `cross_val_score`; `scorers.py` → Scorer default params; `ops.py` → cuDF-compatible operations.

```python
# In scorers.py, modify default XGBoost params:
class XGBScorer(Scorer):
    def __init__(self, **kwargs):
        default_params = {
            "tree_method": "gpu_hist",  # GPU training
            "device": "cuda",
            "n_estimators": 200,
            "learning_rate": 0.1,
            "max_depth": 6,
        }
        default_params.update(kwargs)
        super().__init__(xgb.XGBRegressor(**default_params))

# In ops.py, optional cuDF acceleration:
try:
    import cudf
    def gpu_groupby_mean(df, cat_col, num_col):
        gdf = cudf.from_pandas(df[[cat_col, num_col]])
        return gdf.groupby(cat_col)[num_col].transform("mean").to_pandas()
except ImportError:
    pass
```

NVIDIA benchmarks show cuDF delivers **up to 150× speedup** for groupby operations on A100 GPUs. For the DataCrunch compute environment (10 hours GPU/week), this is the difference between evaluating 5,000 and 500,000 candidates. **Implementation complexity**: 1–2 days.

Priority 7: Temporal operators for DataCrunch (expected impact: HIGH for this competition)

Why: DataCrunch 2 data has weekly "moons" (time periods). Lag features (`feature_value_at_moon_t-1`), rolling statistics (`mean_of_last_4_moons`), and momentum features (`value_at_t - value_at_t-4`) are standard in quant finance and capture temporal dynamics that cross-sectional row-wise ops miss entirely.

Where to modify: `ops.py` → new `TEMPORAL_OPS_LAMBDAS`; `features.py` → child generation logic with temporal awareness.

```python
```

```python
# In ops.py:
TEMPORAL_OPS = {
    "lag_1": lambda df, col, id_col, time_col:
        df.groupby(id_col)[col].shift(1),
    "lag_4": lambda df, col, id_col, time_col:
        df.groupby(id_col)[col].shift(4),
    "rolling_mean_4": lambda df, col, id_col, time_col:
        df.sort_values(time_col).groupby(id_col)[col].transform(
            lambda x: x.rolling(4, min_periods=1).mean()),
    "rolling_std_4": lambda df, col, id_col, time_col:
        df.sort_values(time_col).groupby(id_col)[col].transform(
            lambda x: x.rolling(4, min_periods=1).std()),
    "momentum_4": lambda df, col, id_col, time_col:
        df[col] - df.groupby(id_col)[col].shift(4),
    "pct_change_1": lambda df, col, id_col, time_col:
        df.groupby(id_col)[col].pct_change(1),
}
```

Leakage safety: These ops use `.shift()` and backward-looking `.rolling()`, so they only reference past values. They must be marked `pipeline_required=True` and computed after sorting by time, with proper handling of the first rows (NaN from insufficient history). For DataCrunch 2, the `moon` column is the time key and stock IDs are the group key — but IDs change across moons, so you'd need to map via a cross-reference or use the data's implicit ordering.

Implementation complexity: 2–3 days. Requires adding `time_col` and `id_col` parameters to the feature generation config.

Priority 8: LLM-assisted feature proposal injection (expected impact: MEDIUM)

Why: CAAFE (NeurIPS 2023) and OCTree (NeurIPS 2024) demonstrate that LLMs can propose domain-appropriate features that systematic search misses. For DataCrunch 2's anonymized features this is less useful, but for competitions with named columns (e.g., Zindi's liquidity stress challenge), LLM proposals complement the genetic search.

Where to modify: `features.py` → add `_inject_llm_proposals()` called at generation 0.

```python
```

```
def _inject_llm_proposals(self, column_names, dtypes, task_description):
    """Use an LLM to propose initial feature formulas, seeding gen 0."""
    import openai  # or local LLM
    prompt = f"""Dataset columns: {list(zip(column_names, dtypes))}
    Task: {task_description}
    Propose 20 useful engineered features as Python expressions using column names.
    Format: feature_name = expression
    Only use: +, -, *, /, log, sqrt, abs, and groupby aggregations."""

    response = openai.chat.completions.create(
        model="gpt-4o-mini", messages=[{"role": "user", "content": prompt}]
    )
    # Parse response into Feature candidates and inject into generation[0]
    proposals = self._parse_llm_features(response.choices[0].message.content)
    return proposals
```

For DataCrunch 2's anonymized columns, skip this. For named-column competitions, inject LLM proposals as **seed candidates** in generation 0 alongside the standard random initialization. **Implementation complexity**: 1 day.

Priority 9: Nested CV or regularized wrapper for final selection (expected impact: MEDIUM)

Why: After generating hundreds of features across generations, fitting a single L1-regularized model to select the joint-optimal subset is faster and often better than the greedy forward-selection path that produced them.

Where to modify: `features.py` → add `_final_regularized_selection()` called after `search()` completes.

python

```python
def _final_regularized_selection(self, X_with_generated, y):
    """After search, use L1 regularization to jointly select the best feature subset
    from sklearn.linear_model import LassoCV
    from sklearn.preprocessing import StandardScaler

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X_with_generated)

    lasso = LassoCV(cv=5, alphas=np.logspace(-4, 1, 50), max_iter=10000)
    lasso.fit(X_scaled, y)

    # Keep features with non-zero coefficients
    selected_mask = np.abs(lasso.coef_) > 1e-6
    selected_features = X_with_generated.columns[selected_mask].tolist()

    # Also run tree-based importance as cross-check
    import xgboost as xgb
    xgb_model = xgb.XGBRegressor(n_estimators=300, max_depth=6)
    xgb_model.fit(X_with_generated, y)
    tree_imp = pd.Series(xgb_model.feature_importances_,
                         index=X_with_generated.columns)

    # Final set: union of L1-selected and top-K tree-important
    final = set(selected_features) | set(tree_imp.nlargest(len(selected_features)).i
    return list(final)
```

This catches features that the greedy path missed (jointly valuable but individually weak) and removes features that were accepted early but became redundant as later features were added. **Implementation complexity:** 1 day.

Priority 10: Meta-learning warm start (expected impact: MEDIUM-LOW)

Why: For repeated competition submissions (DataCrunch 2 runs weekly), storing which operators succeeded on past moons and using that to bias initial exploration saves generations of cold-start search.

Where to modify: `features.py` → `ImprovedAdaptiveController`, add serialization methods.

```python
```

```python
# In ImprovedAdaptiveController:
def save_meta_knowledge(self, filepath):
    """Serialize learned op_stats, successful_patterns for future warm-start."""
    meta = {
        "op_stats": dict(self.op_stats),
        "successful_patterns": list(self.successful_patterns),
        "feature_as_parent_success": dict(self.feature_as_parent_success),
    }
    import json
    with open(filepath, "w") as f:
        json.dump(meta, f, default=str)

def load_meta_knowledge(self, filepath):
    """Initialize from previous run's learned knowledge."""
    import json
    with open(filepath) as f:
        meta = json.load(f)
    # Decay old knowledge (50% weight) to allow adaptation
    for key, stats in meta["op_stats"].items():
        self.op_stats[tuple(key)] = {
            k: v * 0.5 for k, v in stats.items()
        }
    self.successful_patterns = meta["successful_patterns"][-10:]  # Keep recent
```

For DataCrunch 2's weekly cadence, last week's op_stats directly inform this week's search — the dataset structure is stable across moons, so operator effectiveness transfers strongly. **Implementation complexity**: 0.5 days.

Priority ranking summary

Priority	Recommendation	Impact	Complexity	Reason
1	FeatureBoost proxy evaluation	HIGH	2–3 days	50× throughput increase; enables exploring vastly larger search space
2	Group-by aggregation operators	HIGH	3–4 days	Missing the single most powerful feature class; critical for DataCrunch panel data
3	CV selection bias fix	MEDIUM-HIGH	1–2 days	Prevents overfitting feature set to validation noise; improves OOS transfer

Priority	Recommendation	Impact	Complexity	Reason
4	Batch multi-feature evaluation	MEDIUM	1–2 days	Discovers complementary features greedy selection misses
5	Feature value caching	MEDIUM	0.5 days	Free speedup with minimal code changes
6	GPU acceleration	MEDIUM	1–2 days	Leverages existing CUDA expertise; large speedup on available hardware
7	Temporal operators	HIGH (DataCrunch)	2–3 days	Captures time dynamics missing from current op set
8	LLM feature injection	MEDIUM	1 day	Useful on named-column datasets; skip for anonymized competitions
9	Regularized wrapper post-selection	MEDIUM	1 day	Better joint optimization than greedy path alone
10	Meta-learning warm start	MEDIUM-LOW	0.5 days	Compounds advantage on weekly competitions like DataCrunch

How the genetic infrastructure compares to 2025 SOTA

The field has moved decisively toward **LLM-guided feature engineering** (OCTree at NeurIPS 2024, LLM-FE and REFeat in 2025 preprints). These methods use LLMs as mutation operators in evolutionary loops — conceptually similar to TabularAML's genetic approach but with natural language reasoning replacing random operator selection. LLM-FE (2025) explicitly combines evolutionary search with LLM-based crossover and mutation, achieving the best published results across diverse benchmarks.

However, these LLM methods have **critical weaknesses** for competitive deployment: they require API costs ($0.50–$5 per search iteration with GPT-4), have high latency (seconds per candidate vs. milliseconds), and struggle with anonymized data (DataCrunch's `gordon_Feature_1` gives the LLM nothing to reason about semantically). TabularAML's purely data-driven genetic search is immune to these limitations.

The most impactful competitive position for TabularAML is as a **hybrid system**: genetic search with FeatureBoost evaluation for systematic numeric/aggregation features (Priorities 1–7), with optional LLM injection for named-column datasets (Priority 8). This would combine OpenFE's

evaluation efficiency, Featuretools' aggregation depth, and TabularAML's existing adaptive search intelligence — with stronger leakage prevention than any of them.

The framework's adaptive stagnation handling, pattern memory, and SHAP-guided parent selection are genuinely novel contributions that no published framework replicates. These become even more valuable when paired with FeatureBoost evaluation, because the increased throughput means the memory and adaptive mechanisms have more data to learn from, creating a compounding advantage over static search strategies.

Conclusion

TabularAML is a **well-engineered framework with strong theoretical foundations** in evolutionary search, leakage prevention, and adaptive exploration. Its main limitations — per-candidate full-CV evaluation cost, missing aggregation operators, and CV selection bias — are all addressable without architectural rewrites. Implementing Priorities 1–3 (FeatureBoost, GroupBy ops, CV bias fix) within ~**1 week of development** would close the gap with OpenFE on evaluation efficiency, surpass it on operator coverage for panel data, and provide stronger statistical guarantees. For DataCrunch 2 specifically, Priorities 2 and 7 (aggregation and temporal operators) target the exact feature classes that quantitative finance alpha signals require. The genetic infrastructure's memory and adaptation mechanisms — already superior to published alternatives — will compound in value as throughput increases unlock more learning iterations per search budget.